

Bike Theft Tracker

Technical Software Documentation

Group 58 | Batch 2022F | BS Computer Science | SSUET, Karachi

Document version: 1.1

Date: 25 April 2026

Audience: Technical reviewers, engineers, evaluators

Source of truth: generated from project source code and repository markdown

1. Project Overview

Bike Theft Tracker (BTT) is a four-role REST platform for reporting, investigating, and recovering stolen motorcycles in Pakistan. It combines a JWT-secured Django REST API with a React (Vite) single-page client and a thin ML layer (RapidFuzz, scikit-learn DBSCAN, PostGIS spatial queries) that wires citizen sightings to active investigations.

Problem solved

- Owners have no centralized way to report a theft and track its lifecycle.
- Police lack tooling to fuzzy-match damaged engine/chassis plates against active cases.
- Communities have no channel to surface suspicious sightings to law enforcement.
- Administrators need national-level analytics on theft hotspots, trends, and recovery rates.

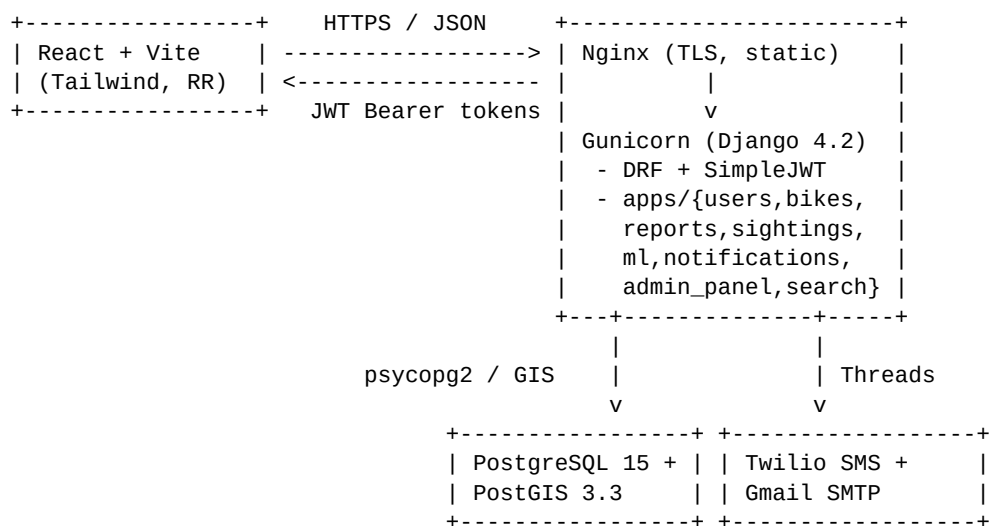
Roles

owner, community, authority (city-scoped), **admin**.

Primary outcomes

- Single audit-logged case lifecycle shared by all roles (new_case → under_review → active_investigation → bike_located → pending_verification → recovered → closed).
- Sub-second fuzzy matching of partial alphanumeric identifiers against the live theft pool.
- DBSCAN-clustered crime hotspots and pandas-aggregated trend analytics for policy decisions.

2. System Architecture



Pattern: classic three-tier monolith with a sharply separated SPA. Async work (notifications) uses fire-and-forget daemon threads (monkeypatched to synchronous in the test suite via `_SyncThread`) instead of a broker, which keeps the deployment surface minimal for an FYP-scale production while preserving non-blocking request paths.

Representative data flow - sighting submission

- Community user POSTs `/api/sightings/` with optional partial engine/chassis number + location.

- View → serializer validates and persists SightingReport; Point (lng, lat) stored as PostGIS geometry.
- View calls `apps.ml.fuzzy_match.find_fuzzy_matches(...)` synchronously.
- If best score $\geq$ `ML_FUZZY_OWNER_ALERT_THRESHOLD` (70), an owner-handshake notification is enqueued on a daemon thread.
- Authority dashboard queries `/api/sightings/?unverified=true`; verification fires another notification.

3. Frontend Architecture

Stack: React 18, Vite, Tailwind CSS, React Router v6, Axios.

Folder layout (btt-frontend/src/)

- **pages/** - route components grouped by role (auth/, owner/, community/, authority/, admin/).
- **components/UI/** - design-system primitives: Button, Modal, Badge, Alert, EmptyState, Spinner, PasswordInput. Re-exported via `index.js` barrel.
- **api/** - one Axios instance + per-domain wrappers (authApi, bikeApi, reportApi, sightingApi, mlApi, adminApi, notificationApi).
- **hooks/** - `useFetch` (loading/data/error/refetch tuple), `useAuth`.
- **utils/constants.js** - `ROLE_COLORS`, `ROLE_LABELS`, status enums kept in lockstep with the backend.

State management

Local `useState` for component-scoped state; `useFetch` for server cache (no Redux/RTK Query). JWTs are stored in `localStorage` and injected by an Axios request interceptor; a response interceptor handles 401 → refresh → retry once, then logout.

Forms with live validation

`RegisterPage` and `UsersPage` (admin Create Authority) use a five-state field state machine (idle | checking | ok | taken | invalid) with 600 ms debounce on change, immediate check on blur, and `lastChecked` ref short-circuiting. The `canSubmit` flag composes both fields:

```
const canSubmit =
  !saving &&
  emailState.state !== FIELD_CHECKING &&
  cnicState.state !== FIELD_CHECKING &&
  emailState.state !== FIELD_TAKEN && emailState.state !== FIELD_INVALID &&
  cnicState.state !== FIELD_TAKEN && cnicState.state !== FIELD_INVALID;
```

Reusable PasswordInput

`forwardRef` component with `eye/eye-off` SVG toggle, `aria-pressed`, `tabIndex={-1}` on the toggle so keyboard tab order remains password → submit. Used across `LoginPage`, `RegisterPage`, `ResetPasswordPage`, and admin `UsersPage`.

4. Backend Architecture

Stack: Django 4.2, DRF 3.14, SimpleJWT 5.3, GeoDjango (PostGIS), RapidFuzz 3.x, scikit-learn 1.3 (DBSCAN), pandas 2.1, Twilio 8.x.

App boundaries (btt-backend/apps/)

App	Responsibility
users	Custom User model with role + city, JWT auth views, email/CNIC availability checks, password reset
bikes	Owner-scoped CRUD; engine/chassis numbers immutable post-create
reports	TheftReport lifecycle, recovery sub-resource, audit log writes
sightings	SightingReport create + verify; auto fuzzy-match on POST
ml	fuzzy_match.py, hotspot DBSCAN, trend analytics, recovery-zone PostGIS query
notifications	In-app + email + SMS dispatch via daemon threads
admin_panel	Admin-only user management, KPI analytics, audit log viewer
search	Public bike/city search

Cross-cutting

- **Permissions** - IsOwner, IsAuthority, IsAdmin, IsAuthorityOrAdmin, plus city-scoped object-level checks for authority operations.
- **Throttling** - DRF ScopedRateThrottle for login (5/15min) and the new availability_check (30/min) scope; default Anon/User throttles as baselines.
- **Soft deletes** - every domain model has deleted_at; queries default-filter deleted_at__isnull=True.
- **Reference numbers** - TheftReport.reference_number is a Python @property (f"BTT-{id:04d}"), not a column; tests must derive pk via int(ref.split('-')[-1]).

5. API Specification

Base URL: `https://<host>/api/`. All non-public endpoints require Authorization: Bearer <access_token>.

Auth - /api/auth/

Method	Path	Auth	Notes
POST	/register/	Public	Owner or Community only - Authority created by Admin
POST	/login/	Public	Throttled 5/15min/IP
POST	/token/refresh/	Public	Rotation + blacklist
POST	/logout/	Auth	Blacklists refresh
POST	/verify-email/{token}/	Public	24h expiry
POST	/forgot-password/	Public	Email reset link
POST	/reset-password/{token}/	Public	Sets new password
GET	/check-email/?email=	Public	Throttled 30/min - {available, valid_format, reason}
GET	/check-cnic/?cnic=	Public	Throttled 30/min - strips -/spaces, requires 13 digits

Reports - /api/reports/

Method	Path	Role	Notes
GET	/	Auth	Role-scoped list (Owner sees own, Authority sees city, Admin sees all)
POST	/	Owner	File a new theft report
GET	/<id>/	Auth	Report detail (role-scoped)
PUT	/<id>/status/	Authority/Admin	Drives the case state machine
DELETE	/<id>/	Admin	Soft-delete
POST	/<id>/recovery/	Authority	Log recovery record
GET	/<id>/recovery/	Auth	Recovery detail

Method	Path	Role	Notes
PUT	/[id]/recovery/	Authority	Amend recovery record
PUT	/[id]/recovery/confirm/	Owner	Owner confirms bike pickup after authority marks recovered. Transition

Other domain endpoints (summary)

- **/api/bikes/** - Owner: GET list, POST create, GET/PUT/DELETE detail. Engine/chassis immutable after create.
- **/api/sightings/** - POST creates and runs fuzzy match server-side; PUT /[id]/verify/ Authority-only triggers owner handshake notification; PUT /[id]/owner-confirm/ Owner-only handles yes/no/not_sure response.
- **/api/ml/** - fuzzy-match, hotspots, trends, recovery-zones, trigger-reanalysis.
- **/api/notifications/** - GET list with unread count, PUT mark-read (single + bulk).
- **/api/admin/** - Users CRUD, authority creation, status updates, analytics, immutable audit log.
- **/api/search/** - Bike search by engine/chassis/plate; city active-report counts.

Response convention: DRF JSON. Errors: {detail: '...'} for permission/auth errors; {<field>: ['...']} for validation. Standard codes: 200 / 201 / 204 / 400 / 401 / 403 / 404 / 429 / 500.

6. Authentication and Authorization

Token model (SimpleJWT, rotation + blacklist)

- Access token: 15 min, Bearer flow.
- Refresh token: 7 days, single-use (rotation enabled), blacklisted on use and on /logout/.

Password storage

Django default PBKDF2_SHA256, 260,000 iterations.

Email verification gate

User.is_email_verified is checked in the LoginSerializer; unverified users cannot exchange credentials for tokens.

Role-based access

```
class IsAuthority(BasePermission):
    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.role == 'authority')
```

Object-level checks add city scoping (e.g. report.bike.owner.city == request.user.city for non-admin authorities).

Inline availability checks

The new /check-email/ and /check-cnic/ endpoints normalize input (email.lower().strip(), CNIC stripped of /spaces) and return a stable contract {available, valid_format, reason} to keep the React state machine deterministic.

7. Database Design

Engine: PostgreSQL 15 + PostGIS 3.3 (port 5433 in local dev).

Core tables (9)

Table	Key relationships	Notable columns
users	self	email (unique), cnic (unique, 13 digits), role, city, is_email_verified, is_active, deleted_at
bikes	owner_id -> users	engine_number, chassis_number, registration_number, photo_url, is_stolen (denormalized)
theft_reports	bike_id -> bikes	status (enum), theft_date, theft_city, theft_location (PostGIS Point), description, deleted_at
recovery_records	theft_report_id -> theft_reports (1:1)	recovered_by_id, recovery_date, recovery_location, condition_notes, chain_of_custody
sightings	sighter_id -> users; matched_bike_id -> bikes	partial_engine, partial_chassis, location (PostGIS), verified, verified_by_id, owner_id
notifications	recipient_id -> users	type (enum), title, message, read, payload (JSON), created_at
audit_logs	actor_id -> users	action, target_model, target_id, before / after (JSON), timestamp - append-only (DENY)
case_timeline	report_id -> theft_reports	event_type (enum), actor_role, actor_id -> users (nullable), note (text), created_at
*_tokens	user_id -> users	token (UUID), expires_at (password reset + email verify)

Spatial indexing

GIST indexes on `theft_reports.theft_location` and `sightings.location` enable `ST_DWithin` for recovery-zone queries. The recovery-zones endpoint uses `GeoDjango __distance_lte=Distance(km=...)` which compiles to PostGIS spatial predicates.

Integrity rules

- Engine/chassis numbers are immutable post-create at the serializer layer (no DB CHECK; enforced by `read_only_fields` on update serializers).
- All deletes are soft (`deleted_at = now()`); audit log is the only table without a soft-delete column - it is genuinely immutable.

8. Core Features

8.1 Theft case lifecycle

```
new_case -> under_review -> active_investigation -> bike_located
          -> pending_verification -> recovered -> closed
                                   ^ legacy: stolen, under_investigation
```

Transitions are validated server-side; only authority (city-scoped) and admin can move the state forward. Every transition writes an `AuditLog` row.

8.2 Fuzzy engine/chassis matching

`apps/ml/fuzzy_match.py::find_fuzzy_matches` runs `RapidFuzz wRatio` against the in-memory candidate dict of all currently active theft reports. Thresholds (settings-overridable):

- `ML_FUZZY_HIGH_THRESHOLD = 85` -> HIGH
- `ML_FUZZY_MEDIUM_THRESHOLD = 70` -> MEDIUM
- below -> LOW

Active-status filter (must mirror `Bike.is_stolen`):

```
ACTIVE_THEFT_STATUSES = [
    Status.STOLEN, Status.UNDER_INVESTIGATION,          # legacy
    Status.NEW_CASE, Status.UNDER_REVIEW,
    Status.ACTIVE_INVESTIGATION, Status.BIKE_LOCATED,
    Status.PENDING_VERIFICATION,
]
```

8.3 Sighting handshake

On sighting submission, fuzzy match runs synchronously. If the score is ≥ 70 (or ≥ 85 with a photo, in which case the authority is also alerted immediately), the bike owner receives an in-app handshake notification with three response options:

- **yes** → sighting flagged as owner-confirmed; URGENT escalation sent to Authority.
- **no** → sighting archived; community reporter notified of the no-match outcome.
- **not_sure** → sighting stays open; auto-escalates to Authority after the timeout window.
- **(timeout)** → no response within the response window → the system nudges the owner at 24 h, then auto-escalates via `auto_escalate_pending_owner_responses()`.

All four branches write a `CaseTimeline` event so the Owner and Authority dashboards can render a complete audit trail of the handshake.

8.4 Hotspot clustering

Management command `run_hotspot_analysis` runs `sklearn.cluster.DBSCAN` (haversine metric, configurable `eps_km` and `min_samples`) on theft locations. Results are persisted as `HotspotCluster` rows for the `/api/ml/hotspots/` read endpoint.

8.5 Trend analytics

`run_trend_analytics` aggregates monthly theft/recovery counts per city via `pandas`, persists to `TrendSnapshot`, and powers admin charts.

8.6 Recovery zones

`/api/ml/recovery-zones/?lat=&lng=&radius_km=` runs PostGIS `ST_DWithin` to surface where bikes stolen near a query point were ultimately recovered.

8.7 Admin user management

Admin creates authority accounts directly (bypassing the public `/register/` whitelist) via `/api/admin/users/authority/`. The form now uses inline email + CNIC availability checks and a show/hide password input identical to the public `RegisterPage`.

9. State Management

Frontend

- Local `useState` for forms.
- `useFetch(fn, deps)` for server reads - returns `{ data, loading, error, refetch }`.
- Auth state in `useAuth` (custom hook) reading from `localStorage`; protected routes wrap with `<RequireAuth role='...'>`.
- Form-level FSMs (`RegisterPage`, admin `UsersPage`) hold per-field validation state outside the form value object so debounced async checks update independently of keystrokes.

Backend

- Stateless REST. All durable state lives in PostgreSQL; JWT carries identity.
- Token blacklist table enforces logout/refresh-rotation.
- Audit log is the canonical source of truth for who did what when - never updated.

10. Error Handling

Backend

- Serializer validation errors auto-raise `ValidationError` -> 400 with field-keyed payload.
- Custom DRF exception handler converts auth failures to `{detail: '...'}` 401, permission failures to 403.
- Unhandled exceptions logged via Python logging; production maps them to a generic 500.
- ML endpoints degrade gracefully - missing `rapidfuzz` returns `[]` and logs `ERROR` rather than crashing the request.

Frontend

- Axios response interceptor: on 401 attempts a single refresh; on second 401 logs out and redirects to `/login`.
- Each page surfaces a single `<Alert type='error'>` block bound to a `useState('')` error string; flattened from `Object.values(data).flat().join(' ')` when the backend returns multi-field validation.
- Inline form fields show their own micro-error (`FieldStatus` for email/CNIC) without dismissing the global alert.

11. Testing

Backend: `pytest-django` with `pytest-cov`. Latest run:

```
===== 335 passed in 264.69s (0:04:24) =====
```

Test files (335 total)

- `test_auth.py` (17), `test_auth_extended.py` (15), `test_admin.py` (25)
- `test_bikes.py` (17), `test_bike_serializers_extra.py` (2)
- `test_reports.py` (20), `test_reports_extended.py` (20), `test_case_timeline.py` (1)
- `test_sightings.py` (30), `test_sighting_handshake.py` (4)
- `test_fuzzy_match.py` (15), `test_ml.py` (33)
- `test_notifications.py` (36)
- `test_security.py` (22), `test_coverage_boost.py` (10)
- `test_enhanced_flows.py` (3)
- `test_inter_role_sync.py` (64) - includes `TestEndToEndDemoNarrative::test_full_demo_scenario_runs_end_to_end` covering all six cross-role events in one assertion-rich integration test.

Coverage: `pytest` configured for `--cov=apps`; project minimum 80%.

Async neutralization: `_SyncThread` monkeypatches `threading.Thread` in test fixtures so notification dispatch executes inline, guaranteeing assertions run after side effects.

Frontend: `Playwright` is installed (`@playwright/test` in `node_modules`); E2E suite is the natural next addition.

12. Performance Considerations

- `select_related('bike', 'bike__owner')` in fuzzy match avoids N+1 across the candidate set.
- In-memory candidate dict keyed by stringified bike id - `RapidFuzz process.extract` is C-accelerated; even with low-thousands candidates the call is sub-50 ms.
- PostGIS GIST indexes on theft/sighting geometries make `ST_DWithin` constant-time on reasonable radii.
- DBSCAN clustering is a batch job (`run_hotspot_analysis`) writing snapshots - read endpoints serve precomputed rows, never recompute on request.
- Daemon-thread notifications keep the request path under 100 ms even when SMTP/Twilio are slow; failures are logged but never propagate.
- JWT removes session lookups from the hot path.
- DRF throttling caps abusive traffic before view code runs (`availability_check`: 30/min, `login`: 5/15min).

13. Deployment

Target: Ubuntu 22.04 LTS, single VPS.

Components

- Nginx 1.24 - TLS termination (Certbot/Let's Encrypt), static asset serving, reverse proxy to Gunicorn.
- Gunicorn 21 - WSGI app server, supervised by supervisor.
- PostgreSQL 15 + PostGIS 3.3 - system service.
- Cron - schedules `run_hotspot_analysis --all-cities` and `run_trend_analytics`.

Files supplied

- `deploy/nginx_btt.conf`
- `deploy/supervisor_btt.conf`
- `deploy/crontab.txt`
- `.env.example`

Provisioning is codified in `README.md` - Production Deployment (`apt deps` -> `DB` + extension -> `clone` + `venv` + `migrate` -> `collectstatic` -> `nginx` -> `certbot` -> `supervisor` -> `cron` -> `seed` + `ML`).

Local dev: `start_dev.bat` boots PostgreSQL on port 5433 and `python manage.py runserver`. `load_dotenv(override=True)` ensures `GDAL_LIBRARY_PATH` from `.env` wins over a stale machine env var.

Hardening steps applied

- **Single-worker Gunicorn** - `--workers 1` pinned in `deploy/supervisor_btt.conf`. Required until Celery replaces the daemon-thread notification dispatch; multi-worker would break the exactly-once notification guarantee.
- **Audit-log DB-level immutability** - migration `users/0002_audit_log_immutability.py` issues `REVOKE UPDATE, DELETE ON audit_logs FROM bttadmin` with a safe `reverse_sql` for rollback. Forensic integrity is now enforced at the database, not just the application layer.

14. Limitations

- **Single-VPS monolith** - no horizontal scaling story; daemon-thread dispatch breaks if there are multiple workers and notifications must be exactly-once.
- **No message broker** - Celery/Redis would replace `_SyncThread` for resilience; current design is acceptable only at FYP scale.
- **Hard-coded city scoping** - authority access is partitioned by free-text city string; no provincial/district hierarchy.
- **Fuzzy match thresholds are global** - WRatio thresholds are not learned per identifier shape (Pakistani plates vary widely).
- **Rate limiting is in-process** - DRF throttle cache is local memory; switching to Redis is required for multi-worker/multi-host deployments.
- **No image OCR** - sightings carry typed partial numbers only; OCR on uploaded plate photos would improve recall.
- **No mobile client** - React SPA is responsive but there is no PWA manifest or native wrapper.
- **Twilio + Gmail SMTP** - fine for demo, not for high-volume production; outbound throughput will become the bottleneck.

15. Future Improvements

Short term (≤ 1 sprint)

- Move notification dispatch behind Celery + Redis; promote `_SyncThread` test pattern to a `CELERY_TASK_ALWAYS_EAGER` config.
- Add Playwright E2E coverage for the six-event demo narrative already covered by `test_inter_role_sync.py`.
- Persist the fuzzy-match audit trail (sighting $\rightarrow$ top candidates $\rightarrow$ decision) for ML retraining.
- Replace `/check-email` and `/check-cnic` with a single `/check-availability/?field=email&value=...` to reduce duplication and let the throttle scope cover both.

Medium term

- OCR plate-photo upload via AWS Textract / Tesseract on the sighting form; auto-populate `partial_engine/partial_chassis`.
- Per-city DBSCAN parameter tuning + a feedback loop that adjusts `eps_km` from observed cluster purity.
- Native push notifications (FCM) alongside email/SMS.
- Provincial/district hierarchy for authority scoping; multi-tenant city packs.
- Database-level audit immutability - REVOKE UPDATE, DELETE ON `audit_logs` FROM `bttadmin` in a dedicated migration.

Long term

- Mobile-first PWA with offline sighting capture (IndexedDB) and background sync.
- Plate-image similarity search (CNN embeddings) to complement WRatio on numeric strings.
- Public anonymized hotspot map for civic transparency, throttled and aggregated to ≥ 500 m precision.
- Multi-tenant deployment per province, with a federated admin role for national analytics.
- SSO with NADRA-issued identity for owners (replacing CNIC self-attestation).

Appendix A - Verified test outcome (this build)

collected 335 items - 335 passed in 264.69s (0:04:24)

Appendix B - Critical fix shipped this iteration

apps/ml/fuzzy_match.py previously filtered only [stolen, under_investigation]. Sightings against any modern-workflow status (active_investigation, bike_located, pending_verification, ...) silently failed to find the bike. Filter expanded to mirror Bike.is_stolen; verified by the new end-to-end test.